C++ COMPLETE REFERENCE LIBRARY • MODULE 11

11 POLYMORPHISM — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 compile time vs runtime polymorphism
3. **Theory Chapter:** 02 upcasting binding and virtual dispatch
4. **Theory Chapter:** 03 object slicing and name hiding
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

11 — Polymorphism

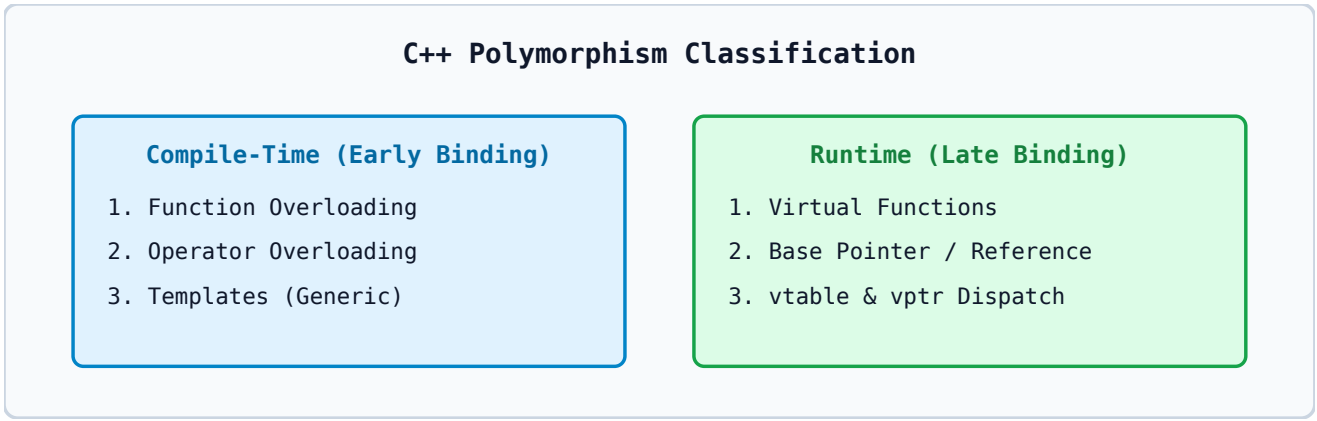
This module explains Compile-Time (Static/Early Binding) vs. Runtime (Dynamic/Late Binding) Polymorphism, function overloading vs. function overriding vs. function hiding, base-class pointers to derived objects (upcasting), and object slicing.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_compile_time_vs_runtime_polymorphism.md` (./01_THEORY/01_compile_time_vs_runtime_polymorphism.md)
  - `02_upcasting_binding_and_virtual_dispatch.md` (./01_THEORY/02_upcasting_binding_and_virtual_dispatch.md)
  - `03_object_slicing_and_name_hiding.md` (./01_THEORY/03_object_slicing_and_name_hiding.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_static_vs_dynamic_binding.cpp` (./02_PROGRAMS/01_static_vs_dynamic_binding.cpp) (Category A)

  - `02_object_slicing_demonstration.cpp` (./02_PROGRAMS/02_object_slicing_demonstration.cpp) (Category A)
  - `03_name_hiding_compile_error.cpp` (./02_PROGRAMS/03_name_hiding_compile_error.cpp) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_polymorphism_classification.svg` (./05_DIAGRAMS/diagram_01_polymorphism_classification.svg)

01 — Compile-Time vs. Runtime Polymorphism



Polymorphism Classification

1. Taxonomy of Polymorphism in C++

1. **Compile-Time Polymorphism (Static Binding / Early Binding)**:
   - Resolved by the compiler at compile time based on static types.
   - Achieved via **Function Overloading**, **Operator Overloading**, and **Templates**.
2. **Runtime Polymorphism (Dynamic Binding / Late Binding)**:
   - Resolved at runtime based on the actual dynamic type of the object pointed/referred to.
   - Achieved via **Inheritance +** `virtual` **Member Functions + Base Pointer or Base Reference**.

02 — Upcasting (`super* ptr = &sub_obj;`) and Static vs. Dynamic Binding

When a base class pointer points to a derived class object:

CPP

```
super* ptr;
sub obj;
ptr = &obj; // Upcasting (always safe and implicit under public inheritance)
ptr→show();
```

- **If `super::show()` is NOT `virtual` (Static Binding)**: The compiler looks only at the pointer's static type (`super*`) and calls `super::show()` .
- **If `super::show()` IS `virtual` (Dynamic Binding)**: The program looks up the object's vtable at runtime and calls `sub::show()` .

03 — Object Slicing and Function Name Hiding

1. Object Slicing

If you assign or pass a `derived` object **by value** into a `base` variable (`base b = d;`), only the `base` subobject portion of `d` is copied; all `derived`-specific data members and the `derived` vptr are **sliced off**! Runtime polymorphism **never** works when passing objects by value — you must use a pointer (`base*`) or reference (`base&`).

2. Function Name Hiding (`using base::func;`)

If class `super` has `void show(int x)` and class `sub` defines `void show()` (with 0 parameters), `sub::show()` **hides** all overloaded versions of `show` in `super` ! Calling `sub_obj.show(10)` fails to compile unless `sub` includes `using super::show;` .

Visual Concept & Memory Diagrams (11 POLYMORPHISM)

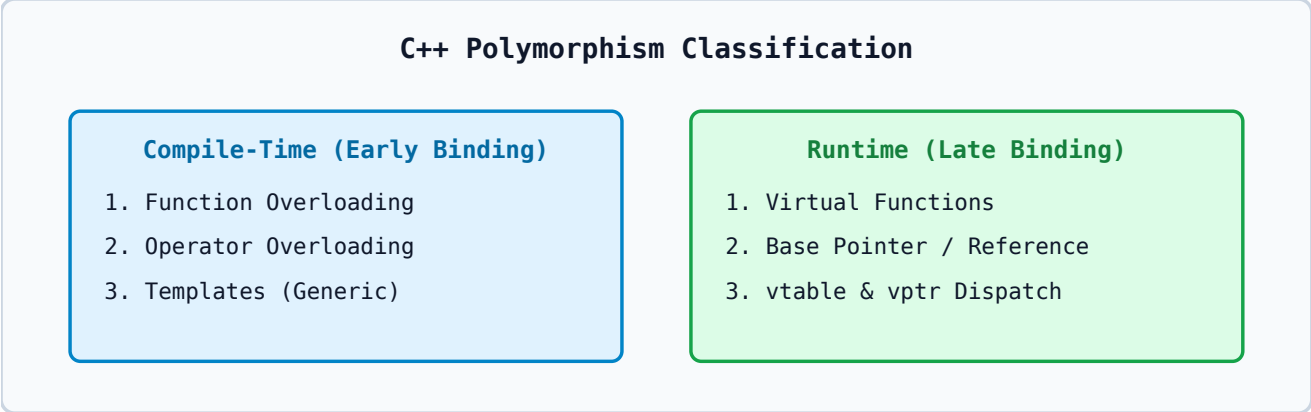


Diagram: diagram 01 polymorphism classification

Annotated C++ Programs (11 POLYMORPHISM)

Program: `01_static_vs_dynamic_binding.cpp`

C++ SOURCE FILE • 01_static_vs_dynamic_binding.cpp

```cpp
// Program Title: Static Binding (Non-Virtual) vs. Dynamic Binding (Virtual)
// Topic: Polymorphism — Upcasting and Late Binding
// Purpose: Compare calling a non-virtual function vs. a virtual function through a base class pointer.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_upcasting_binding_and_virtual_dispatch.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class super {
public:
    void static_show() {
        cout << "super static_show (Early Binding)" << endl;
    }

    virtual void dynamic_show() {
        cout << "super dynamic_show" << endl;
    }

    virtual ~super() = default;
};

class sub : public super {
public:
    void static_show() {
        cout << "sub static_show" << endl;
    }

    void dynamic_show() override {
        cout << "sub dynamic_show (Late Binding)" << endl;
    }
};

int main() {
    sub obj;
    super* ptr = &obj;

    ptr->static_show();  // Calls super::static_show()
    ptr->dynamic_show(); // Calls sub::dynamic_show()

    return 0;
}

/*
Expected Deterministic Output:
super static_show (Early Binding)
```

```
sub dynamic_show (Late Binding)
*/
```

Program: `02_object_slicing_demonstration.cpp`

C++ SOURCE FILE • 02_object_slicing_demonstration.cpp

```cpp
// Program Title: Object Slicing When Passing Derived Object by Value vs. by Reference
// Topic: Polymorphism — Object Slicing
// Purpose: Show how passing by value slices off derived overrides, while passing by const reference preserves polymorphism.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_object_slicing_and_name_hiding.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
public:
    virtual void role() const {
        cout << "Role: General Person" << endl;
    }
    virtual ~person() = default;
};

class student : public person {
public:
    void role() const override {
        cout << "Role: Student" << endl;
    }
};

void print_by_value(person p) {
    p.role(); // Sliced! Always prints General Person
}

void print_by_reference(const person& p) {
    p.role(); // Polymorphic! Prints Student
}

int main() {
    student s;
    print_by_value(s);
    print_by_reference(s);
    return 0;
}

/*
Expected Deterministic Output:
Role: General Person
Role: Student
*/
```

Program: `03_name_hiding_compile_error.cpp`

C++ SOURCE FILE • 03_name_hiding_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Derived Class Member Function Hiding Base Class Overloads
// Topic: Polymorphism — Name Lookup and Hiding in Inheritance
// Purpose: Show why defining show() in sub hides show(int) in super during unqualified
lookup.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/03_object_slicing_and_name_hiding.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class super {
public:
    void show(int x) {
        cout << "super show(int): " << x << endl;
    }
};

class sub : public super {
public:
    void show() {
        cout << "sub show()" << endl;
    }
};

int main() {
    sub obj;
    obj.show(10); // COMPILE ERROR: no matching function for call to 'sub::show(int)'
    return 0;
}

/*
Why Compilation Fails:
Name lookup for 'obj.show' stops as soon as it finds 'sub::show()' inside 'class sub'.
Overload resolution does NOT automatically search base classes once a matching name is found
in the derived scope!
Fix: Add 'using super::show;' inside the 'public:' section of 'class sub', or call
'obj.super::show(10);'.
*/
```

Conceptual Questions — Module 11: Polymorphism

Q1. Compare Function Overloading and Function Overriding across 5 dimensions (inheritance requirement, signature, `virtual` keyword, binding time, and scope). **Q2.** What is Object Slicing and how do you prevent it?

Debugging Questions — Module 11

Q4. How do you unhide overloaded base functions when a derived class defines a function with the same name?

Predict the Output Questions — Module 11

Q3. Predict the output when a base pointer `super* p = &sub_obj;` calls a non-virtual function vs. a `virtual` function.

Programming Questions — Module 11

Q5. Write a C++23 classroom program with base class `shape` (`virtual void area()`) and derived classes `rectangle` and `triangle` invoked polymorphically via `shape*` .

Detailed Solutions — Conceptual Questions (Module 11)

- **Q1 Solution**: Overloading happens in the same scope with different parameter lists and is resolved at compile time. Overriding happens between a base class and a derived class with the exact same signature (`virtual` in base, `override` in derived) and is resolved at runtime via the vtable.
- **Q2 Solution**: Object slicing occurs when a derived object is copied by value into a base object (`base b = d;`), discarding derived data members and resetting the vptr to `base` . Prevent it by passing/referencing via `base*` or `const base&` .

Detailed Solutions — Predict the Output & Debugging (Module 11)

- **Q3 Solution**: Non-virtual call executes `super`'s function (early binding on pointer type `super*`); virtual call executes `sub`'s function (late binding on target object type `sub`).
- **Q4 Solution**: Add `using BaseClassName::function_name;` inside the derived class definition.

Detailed Solutions — Programming Questions (Module 11)

CPP

```cpp
#include <iostream>
using namespace std;

class shape {
public:
    virtual void area() const { cout << "Generic shape
"; }
    virtual ~shape() = default;
};

class rectangle : public shape {
    int l, b;
public:
    rectangle(int x, int y) : l(x), b(y) {}
    void area() const override { cout << "Rectangle area: " << (l * b) << endl; }
};

int main() {
    rectangle r(5, 4);
    shape* s = &r;
    s->area();
    return 0;
}
// Expected Output: Rectangle area: 20
```